

ZKTrustLLM-Agents Level 4

Integrated Full Technical Documentation and Agentic AI Automation Roadmap

Generated: 2026-05-09 14:48

1. Executive Summary

This report integrates the earlier ZKTrustLLM-Agents L4 foundation with the later MCP/A2A, media-plane, DTLS-RTP, impairment, and network namespace validation steps. It is intended as a supervisor-ready full-picture document and as a planning base for the next stage of agentic AI automation.

2. Full Project Picture

Layer	Implemented Work	Research Meaning
Control-plane semantics	Roles, policies, actions, trust states, gateway scheduler	Defines the bounded meaning of L4 decisions
Contract state	AgentRegistry, CapabilityManager, PolicyRegistry	Creates authenticated control-plane state
Proof pipeline	AUTH_V2, AUTH_V2.1, AUTH_V2.2	Enables Groth16 proof-governed admissible decisions
MCP/A2A coordination	MCP server, A2A references, bundle verification	Allows agents to coordinate through compact authenticated references
Control telemetry	Semi-live MCP/A2A latency and multi-agent scaling	Measures coordination cost and scalability
Media-plane baseline	Live RTP telemetry	Connects control-plane research to packet-level media delivery
Secure media-plane	DTLS-wrapped RTP validation	Adds protected media delivery baseline
Network stress	Loopback to netem impairment	Measures behaviour under delay, jitter, loss
Namespace realism	Two Linux network namespaces over veth	Moves closer to two-node deployment while staying reproducible
Automation roadmap	Validator, orchestrator, report compiler, feedback parser	Prepares full agentic AI-assisted research workflow

3. Earlier Foundation: AUTH_V2 to AUTH_V2.2

The earlier foundation implemented the L4 control-plane contracts and the progressive zero-knowledge proof pipeline. AUTH_V2 bound the request to control-plane fields. AUTH_V2.1 added policy admissibility. AUTH_V2.2 added trust-state/action admissibility for the concrete Restricted -> Isolate path.

Proof Stage	Added Capability	Strongest Claim
AUTH_V2	Control-plane request binding	Proof binds agent, capability, policy, action, context, expiry, and trace context
AUTH_V2.1	Policy admissibility flag	Proof verifies only when policyAdmissibilityFlag == 1
AUTH_V2.2	Trust-state/action pair	Proof validates Restricted -> Isolate as a bounded admissible action

4. Later Level 4 Progress: Steps 77-93

Step	Main Output
77	A2A reference-aware coordination
79	Proper MCP L4 context server
80	MCP/A2A KPI analysis
81	AUTH_V2.3/reference-bound proof direction

Step	Main Output
82	Negative-security validation
83	Deterministic network KPI telemetry
84	Journal-ready evaluation write-up
85	Benchmark result figures
86	Semi-live MCP/A2A control telemetry
87	Multi-agent scaling telemetry
88	Technical report PDF for Steps 77-87
89	Live RTP media-plane telemetry
90	DTLS-wrapped RTP media-plane validation
91	Plain RTP vs DTLS-RTP network impairment
92	Updated technical report through Step 91
93	Linux network namespace plain RTP vs DTLS-RTP validation

5. Supervisor Feedback Alignment

Supervisor Need	Implemented Evidence	Status
Explain MCP/A2A over L4	Steps 79-87 documentation and telemetry	Done
Use agent KPIs	Step 86 and Step 87 control-plane measurements	Done
Use network KPIs	Steps 89-93 RTP/DTLS/impairment results	Done
Move beyond emulation	Semi-live control telemetry and live RTP validation	Done
Add secure media validation	DTLS-wrapped RTP tunnel/proxy baseline	Done
Test under stress	tc netem loopback and namespace experiments	Done
Prepare for future automation	Step 95 automation roadmap	In progress

6. Current Research Claim

The project now supports a coherent full-stack Level 4 prototype in which proof-governed agent decisions can be represented through contract-backed control-plane state, exchanged between agents through MCP/A2A reference coordination, measured through semi-live control-plane KPIs, and connected to RTP/DTLS media-plane behaviour under controlled network impairment.

7. Forward Agentic AI Automation Plan

Automation Step	Target Script	Purpose
Step 96	validate_l4_results.py	Validate all JSON, CSV, packet, bitrate, jitter, and report artifacts
Step 97	run_l4_full_experiment_suite.py	Run all reproducible experiments from one command
Step 98	supervisor_feedback_tracker.py	Map feedback to project artifacts and status
Step 99	build_l4_master_report.py	Automatically compile Markdown/PDF/paper tables
Step 100	run_two_machine_or_mininet_validation.py	Move beyond single-machine namespace topology

8. Recommended Next Steps

The next engineering step should be Step 96: a full result validator. This should check that all major result files exist, JSON files parse successfully, packet counts are positive, bitrate is positive, jitter is within realistic bounds, Git status is clean, and the latest PDF report exists.

After that, Step 97 should create a full experiment-suite runner that can execute the control-plane, media-plane, impairment, and namespace workflows in a structured order. This will make the project easier to reproduce and easier to present in supervisor meetings.

9. Practical Continuation Command

Recommended continuation point:

```
cd ~/zktrustllm-agents-l4
git checkout l4-step95-integrated-history-automation-report
source .venv/bin/activate
python scripts/l4/build_l4_integrated_history_automation_report_pdf.py
git status
```